JAVA INTERVIEW PREPARATION

CHAPTER 5

EXCEPTION DESIGN:
FAILURE CONTRACTS AND RECOVERY

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Java Exception Design: Failure Contracts, Recovery, and Production Diagnosis

Senior / Lead Java Developer Reading Chapter

Exceptions are part of a Java API's behavioral contract. A method communicates not only what it returns when successful, but also which failures callers can distinguish, which failures are recoverable, and which context will survive when an operation crosses architectural boundaries.

Poor exception handling usually falls into one of two extremes. Some code catches everything and hides the original problem. Other code catches nothing and leaks low-level implementation details through every layer. Senior-level design sits between those extremes: preserve evidence, translate failures where abstraction changes, recover only when recovery is meaningful, and make operational behavior explicit.

The Exception Hierarchy

All Java exceptions and errors derive from `Throwable`.

```
Throwable
   |
   +-- Error
   |
   +-- Exception
          |
          +-- RuntimeException
          |
          +-- other checked exceptions
```

`Error` represents serious conditions such as `OutOfMemoryError`, `StackOverflowError`, or linkage failure. Applications generally should not catch `Error` as ordinary business failure. Catching it does not restore corrupted capacity or make the JVM healthy.

`Exception` represents conditions application code may encounter. Subclasses of `RuntimeException` are unchecked: the compiler does not require callers to declare or catch them. Other Exception subclasses are checked and participate in compile-time handling rules.

The hierarchy does not automatically tell the application whether retrying is safe or whether a user should see a 400 or 500 response. Those decisions depend on semantics and context.

Checked and Unchecked Exceptions

A checked exception makes a possible failure visible in the method signature.

```
public CustomerFile load(Path path) throws IOException {
    // read and parse the file
}
```

The caller must catch `IOException` or propagate it. This is useful when the condition is a meaningful part of the contract and callers can choose a response such as another file, a user prompt, or an aborted batch.

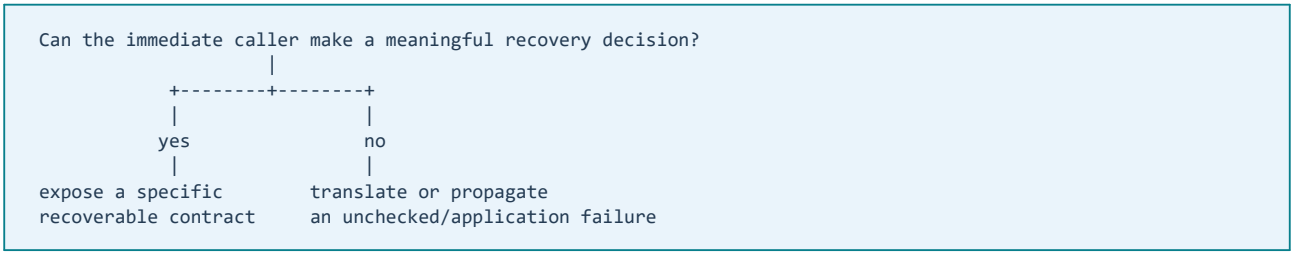
Unchecked exceptions are commonly used when a caller cannot reasonably recover at that level, when a programming contract was violated, or when forcing declarations through many layers would add ceremony without useful decisions.

```
public Money withdraw(Money amount) {
    if (amount.isNegativeOrZero()) {
        throw new IllegalArgumentException(
            "Withdrawal amount must be positive"
        );
    }

    // domain operation
}
```

The checked-versus-unchecked decision should not be reduced to "expected versus unexpected." A database timeout may be expected operationally but not recoverable by the immediate caller. Invalid user input may be expected but should be modeled as a domain or validation failure rather than a generic runtime error.

A useful decision sequence is:



Library APIs may favor checked exceptions where callers operate close to the failing resource. Service architectures often translate low-level checked failures into stable application exceptions at a boundary.

Exceptions Should Describe Failed Operations

An exception is more useful when it explains which contract could not be fulfilled.

Compare a low-level leak:

```
public Order load(OrderId id) throws SQLException {
    return repository.load(id);
}
```

The service contract now exposes the storage technology. If JDBC is replaced with a document database or remote service, callers still depend on `SQLException`.

A boundary-specific exception protects the abstraction:

```
public Order load(OrderId id) {
    try {
        return repository.load(id);
    } catch (SQLException ex) {
        throw new OrderAccessException(
            "Unable to load order " + id.safeValue(),
            ex
        );
    }
}
```

The service describes the failed operation while preserving the original cause. Callers depend on a stable application concept, and production diagnostics retain the database evidence.

Translation should occur when the abstraction changes. Wrapping an exception at every method adds noisy stack layers without adding meaning.

Preserve the Cause

This is a destructive translation:

```
catch (SQLException ex) {
    throw new OrderAccessException("Unable to load order");
}
```

The vendor error code, SQL state, driver message, and original stack are lost.

The exception should accept and retain the cause:

```
public final class OrderAccessException extends RuntimeException {

    public OrderAccessException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

Then translate with:

```
throw new OrderAccessException(
    "Unable to load order " + id.safeValue(),
    ex
);
```

Context must be useful and safe. Include the operation, a non-sensitive identifier, dependency name, or relevant state transition. Do not place passwords, tokens, complete payment details, personal data, or raw request bodies in exception messages.

The exception cause chain should read like a causal narrative:

```
OrderAccessException: unable to load order O-42
       caused by
SQLTransientConnectionException: connection acquisition timed out
       caused by
SocketTimeoutException: database endpoint did not respond
```

Catch Only What You Can Handle

A catch block should perform a meaningful action: recover, translate, compensate, add safe context, or terminate a boundary with a defined response.

This block hides failure:

```
try {
    process(order);
} catch (Exception ex) {
    log.warn("Processing failed");
}
```

The caller may assume processing succeeded. The log loses the stack trace, and the system can continue with incomplete state.

This version is also problematic:

```
try {
    process(order);
} catch (Exception ex) {
    log.error("Processing failed", ex);
    throw ex;
}
```

If an outer boundary also logs the failure, one event produces duplicate logs at several layers. Log once where the operation has enough context and ownership, unless an intermediate layer records a distinct event rather than repeating the stack.

Catch the narrowest exception that supports the intended decision:

```
try {
    paymentGateway.authorize(request);
} catch (PaymentRejected ex) {
    return PaymentResult.rejected(ex.reason());
} catch (GatewayUnavailable ex) {
    throw new CheckoutUnavailableException(ex);
}
```

Business rejection and technical unavailability have different semantics and should not be collapsed into one generic failure.

Do Not Use Exceptions for Ordinary Control Flow

Exceptions carry stack and diagnostic behavior intended for exceptional paths. They also obscure the normal path when used as branching logic.

```
try {
    return customers.get(id).name();
} catch (NullPointerException ex) {
    return "Unknown";
}
```

This catches any null dereference in the expression, not merely a missing customer. It can hide a programming defect.

Use the domain operation directly:

```
return Optional.ofNullable(customers.get(id))
    .map(Customer::name)
    .orElse("Unknown");
```

Exceptions remain appropriate when the operation cannot fulfill its contract. The point is not performance alone; it is semantic clarity.

Try-With-Resources

Resources such as streams, readers, JDBC statements, and result sets must be released even when work fails.

```
try (BufferedReader reader = Files.newBufferedReader(path)) {
    return reader.lines().toList();
}
```

The resource must implement `AutoCloseable`. Java closes resources in reverse declaration order.

```
try (Connection connection = dataSource.getConnection();
     PreparedStatement statement = connection.prepareStatement(sql);
     ResultSet results = statement.executeQuery()) {

    return mapResults(results);
}
```

If the body throws and `close()` also throws, Java preserves the body exception as primary and attaches the close failure as a suppressed exception.

```
 Primary failure: parsing the file failed
        |
        +-- suppressed: closing the stream also failed
```

Suppressed exceptions are available through `getSuppressed()`. Logging frameworks normally display them with the stack trace. Manual `finally` code can accidentally replace the original failure with the close failure, which is one reason try-with-resources is preferred.

Multiple Catch and Precise Rethrow

When several exception types require the same handling, multi-catch avoids duplication:

```
try {
    importFile(path);
} catch (IOException | ParseException ex) {
    throw new ImportException(
        "Unable to import " + path.getFileName(),
        ex
    );
}
```

The alternatives cannot have a parent-child relationship because the broader type would make the narrower alternative redundant.

Java can also infer precise rethrow types in some cases:

```
void execute() throws IOException, ParseException {
    try {
        performImport();
    } catch (Exception ex) {
        auditFailure(ex);
        throw ex;
    }
}
```

If the catch parameter is effectively final and the try block can throw only the declared alternatives, the compiler can preserve those types. Although valid, catching a broad type should still have a clear purpose and should not become a habit.

Custom Exception Design

A custom exception is useful when callers need to distinguish a stable failure category or when a boundary needs domain-specific context.

```
public final class InsufficientFundsException
        extends RuntimeException {

    private final AccountId accountId;
    private final Money requested;
    private final Money available;

    public InsufficientFundsException(
            AccountId accountId,
            Money requested,
            Money available) {

        super("Insufficient funds for account "
            + accountId.safeValue());

        this.accountId = accountId;
        this.requested = requested;
        this.available = available;
    }
}
```

Structured fields let an error mapper produce a response without parsing the message. The message remains useful for diagnostics.

Avoid creating a separate exception class for every line of code. Useful categories align with decisions: not found, rejected, conflict, invalid state, unavailable dependency, or corrupted input. A taxonomy that is too broad forces callers to inspect messages; one that is too granular couples callers to implementation details.

Exceptions should normally be immutable. If they may cross serialization boundaries, treat serialized exception classes as compatibility-sensitive and avoid exposing internal Java exception graphs directly to clients.

Boundary Mapping in REST Services

Internal exceptions should be mapped to a stable external error contract.

```
@RestControllerAdvice
public final class ApiExceptionHandler {

    @ExceptionHandler(OrderNotFoundException.class)
    ResponseEntity<ProblemDetail> notFound(
            OrderNotFoundException ex) {

        ProblemDetail problem =
            ProblemDetail.forStatus(HttpStatus.NOT_FOUND);

        problem.setTitle("Order not found");
        problem.setProperty("code", "ORDER_NOT_FOUND");
        problem.setProperty("traceId", currentTraceId());

        return ResponseEntity.status(404).body(problem);
    }
}
```

The response exposes a stable code, safe description, HTTP status, and correlation identifier. It does not expose stack traces, database messages, internal class names, or secrets.

A validation failure is normally a 4xx response. An unexpected defect or unavailable internal dependency is normally a 5xx response. Authentication and authorization failures have separate security semantics. Status selection should follow the external contract, not simply the exception's Java inheritance.

Retryability Is Not an Exception-Class Guess

Not every technical exception is retryable, and not every retry is safe.

A timeout may mean the remote operation did not execute, or it may mean the response was lost after the operation completed. Retrying a non-idempotent payment can duplicate the charge.

Retry decisions require:

- A transient failure category.
- An idempotent operation or idempotency key.
- A total deadline and bounded attempts.
- Backoff and jitter.
- Awareness of downstream overload.
- Metrics showing attempts and final outcomes.

```
try {
    return retry.execute(() -> client.fetchCustomer(id));
} catch (RetryExhaustedException ex) {
    throw new CustomerServiceUnavailable(id, ex);
}
```

The final exception should preserve the attempt history or last cause while presenting a stable application failure.

InterruptedException and Cancellation

Interruption is a cooperative cancellation signal. Code that catches `InterruptedException` must not silently discard it.

```
try {
    queue.put(job);
} catch (InterruptedException ex) {
    Thread.currentThread().interrupt();
    throw new JobSubmissionInterruptedException(ex);
}
```

The interrupted status is cleared when `InterruptedException` is thrown. Restoring it allows higher layers and executors to observe cancellation.

If the current method can declare `InterruptedException`, propagating it directly is often clearer. If a framework boundary requires translation to an unchecked exception, restore the flag before translating.

Treating interruption as an ordinary retryable error can prevent shutdown, keep locks or resources active, and make thread-pool exhaustion worse.

Exceptions in Asynchronous Code

Exceptions thrown inside `CompletableFuture` stages are captured and represented by exceptional completion.

```
CompletableFuture<Customer> future =
    CompletableFuture.supplyAsync(
        () -> customerClient.load(id),
        executor
    );

return future
    .orTimeout(800, TimeUnit.MILLISECONDS)
    .handle((customer, failure) -> {
        if (failure == null) {
            return CustomerResult.available(customer);
        }

        Throwable cause = unwrapCompletionFailure(failure);
        return mapFailure(cause);
    });
```

`join()` commonly throws `CompletionException`, while `get()` uses checked `ExecutionException`. The original failure is available as the cause. Repeatedly wrapping completion exceptions without unwrapping can obscure the useful root.

An exceptionally completed stage is not automatically logged. The application must eventually observe, return, or record it. Fire-and-forget futures can lose failures unless they attach a terminal handler.

Assertions, Validation, and Exceptions

Java assertions are disabled by default unless enabled at runtime. They are useful for internal assumptions during development but should not enforce required business validation or public preconditions.

```
assert total.signum() >= 0 : "total must not be negative";
```

If the rule must always be enforced, validate explicitly and throw an appropriate exception.

```
if (total.signum() < 0) {
    throw new IllegalArgumentException(
        "Total must not be negative"
    );
}
```

Use `IllegalArgumentException` when a caller violates a method precondition, `IllegalStateException` when the receiver cannot perform the operation in its current state, and domain-specific exceptions when callers need a stable business distinction.

Logging Exceptions Correctly

A production log should preserve the exception object, not only its message.

```
log.error(
    "order_processing_failed orderId={} traceId={}",
    orderId.safeValue(),
    traceId,
    ex
);
```

Passing `ex` in the exception position records its type, message, cause chain, suppressed failures, and stack.

Avoid this:

```
log.error("Failure: " + ex.getMessage());
```

It discards the stack and can create unnecessary concatenation. Do not log the same exception at every layer. Choose the boundary that owns the failed operation and has the most useful safe context.

Expected business outcomes may not require error-level stack traces. A rejected payment or missing optional record can be represented through metrics and structured outcome logs. Reserve error logs for conditions requiring operational attention according to the service's observability policy.

Testing Failure Behavior

Tests should verify the contract, not merely that "some exception" occurs.

```
@Test
void withdrawalRejectsAmountAboveBalance() {
    BankAccount account = fundedAccount("100.00");

    InsufficientFundsException failure = assertThrows(
        InsufficientFundsException.class,
        () -> account.withdraw(money("150.00"))
    );

    assertEquals(money("150.00"), failure.requested());
    assertEquals(money("100.00"), failure.available());
}
```

Boundary tests should confirm translation and cause preservation:

```
assertThatThrownBy(() -> service.load(orderId))
    .isInstanceOf(OrderAccessException.class)
    .hasCauseInstanceOf(SQLException.class);
```

Also test resource closure, retry limits, interruption propagation, and external error responses. Failure-path tests often reveal more architectural coupling than happy-path tests.

Production Perspective

Exception problems in production are often information problems. The service returns a generic 500 while the cause was discarded. Every layer logs the same stack and overwhelms the logging system. A retry wrapper hides the first failure and multiplies dependency traffic. An interrupted worker continues running during shutdown. A close failure replaces the parsing failure that actually mattered.

When investigating a failure:

- Start from the first causal exception, not only the outer wrapper.
- Inspect causes and suppressed exceptions.
- Group failures by stable exception type and error code.
- Correlate the trace ID with dependency latency and pool metrics.
- Compare the failure with recent releases and configuration changes.
- Check whether retries multiplied one user request into many dependency calls.
- Confirm that logs contain safe identifiers but no secrets.
- Preserve evidence before restarting when operationally safe.

An exception taxonomy should support these operations. If every failure becomes `RuntimeException`, grouping is weak. If every infrastructure detail escapes the service, contracts are unstable. The useful middle ground exposes stable categories and preserves detailed causes internally.

Interview-Ready Summary

Exceptions are part of an operation's contract. Checked exceptions are useful when callers can make a meaningful recovery decision and the failure belongs in the explicit API. Unchecked exceptions are appropriate for violated preconditions, programming errors, or failures that callers cannot usefully recover from at that level.

Catch an exception only to recover, translate, compensate, add safe context, or terminate a boundary with a defined response. Translate where abstraction changes and always preserve the original cause. Avoid catch-all suppression, duplicate logging, and exceptions as normal control flow.

Try-with-resources guarantees closure and preserves body failures while attaching close failures as suppressed exceptions. Custom exceptions should represent stable decision categories and may expose structured safe fields rather than requiring message parsing.

REST boundaries should map internal exceptions to stable, non-sensitive external errors. Retryability requires transient classification, idempotency, bounded attempts, deadlines, and backoff. `InterruptedException` is a cancellation signal and must be propagated or restored before translation.

Asynchronous exceptions require explicit observation and cause unwrapping. Production-quality handling preserves causal evidence, supports grouping and correlation, and makes the external contract independent of internal implementation technology.

Revision Notes

- Exceptions are part of a method's behavioral contract.
- `Error` is not an ordinary application-recovery mechanism.
- Use checked exceptions when immediate callers can make meaningful recovery decisions.
- Use unchecked exceptions for violated contracts or failures not recoverable at that level.
- Catch only to recover, translate, compensate, add context, or define a boundary response.
- Translate failures where the abstraction changes, not at every method.
- Always preserve the original cause when wrapping.
- Add useful safe context; never expose secrets or sensitive payloads.
- Do not swallow exceptions or log only their messages.
- Avoid logging the same failure at every layer.
- Do not use exceptions for ordinary branching or null detection.
- Try-with-resources closes in reverse order and retains close failures as suppressed exceptions.
- Custom exceptions should represent stable categories callers can act on.
- REST error responses should contain stable codes, safe descriptions, and correlation IDs.
- Retry only transient and safe operations with deadlines, limits, backoff, and jitter.
- Restore the interrupt flag when translating `InterruptedException`.
- Observe exceptional completion in asynchronous operations.
- Assertions do not replace mandatory validation.
- Test exception type, structured fields, cause preservation, resource cleanup, and boundary mapping.
- Diagnose production failures through cause chains, suppressed exceptions, traces, and dependency metrics.